

# SMART METRO TICKET GENERATING SYSTEM

## Project Planning Template

(Product Backlog, Sprint Planning, Stories, Story Points)

### 1. Product Backlog

The Product Backlog is the single, ordered list of everything known to be needed in the Smart Metro Ticket Generating System, ranked by priority. It is never complete — items are continuously added, removed, and re-prioritized as the team learns more from stakeholders and each sprint's feedback. Every item traces back to a user story defined in the Data Flow Diagrams and User Stories document, so the backlog stays grounded in evidenced commuter and operator needs rather than speculative features.

The flow below shows how large capability areas (epics) are refined into individually estimated stories, pulled into a sprint, built, and fed back into re-prioritization — the same loop introduced in the Project Planning Phase document, now expressed at the backlog level.

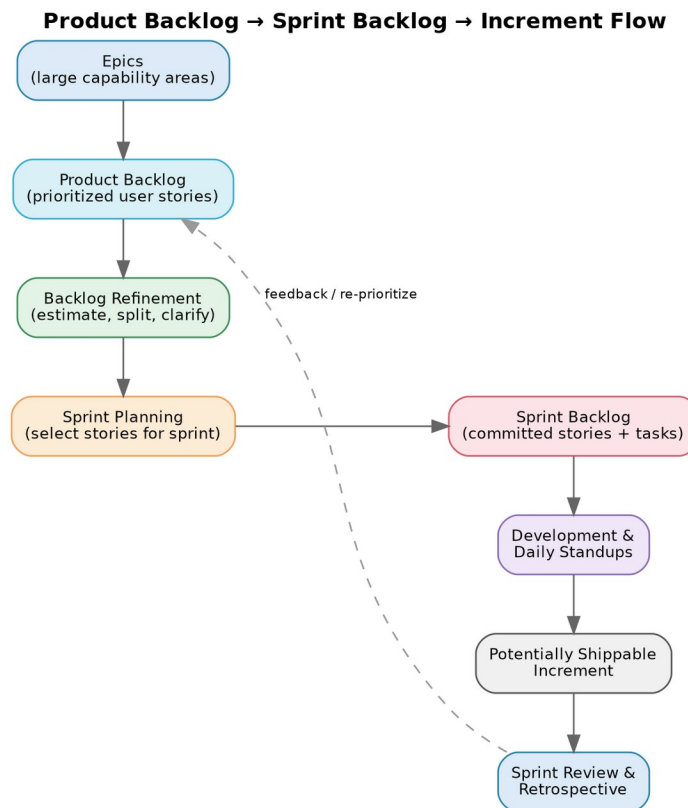


Figure 1: Product backlog to sprint backlog to increment flow, with feedback re-prioritization

## 2. Sprint Planning and Backlog Snapshot

Sprint Planning is the ceremony where the team selects the highest-priority backlog items it can realistically complete in the next sprint, based on its measured velocity from prior sprints. The table below is a snapshot of the backlog across the first three sprints, showing each story's estimate and the sprint it was assigned to.

| ID    | User Story                                  | Points | Sprint   |
|-------|---------------------------------------------|--------|----------|
| US-01 | Buy single-journey ticket via app/web/kiosk | 5      | Sprint 1 |
| US-02 | See fare before payment confirmation        | 2      | Sprint 1 |
| US-03 | Tap phone/NFC at gate to enter              | 8      | Sprint 1 |
| US-04 | Recharge smart card / wallet remotely       | 5      | Sprint 2 |
| US-05 | Station staff gate override with audit log  | 3      | Sprint 2 |
| US-06 | Admin fare-rule configuration screen        | 8      | Sprint 2 |
| US-07 | Daily revenue report by station             | 5      | Sprint 3 |
| US-08 | Refund request before journey start         | 5      | Sprint 2 |

### 2.1 Scrum Ceremony Cycle

Sprint Planning is one of five recurring ceremonies that structure every sprint. The diagram below shows the full cycle the team follows for each two-week sprint.

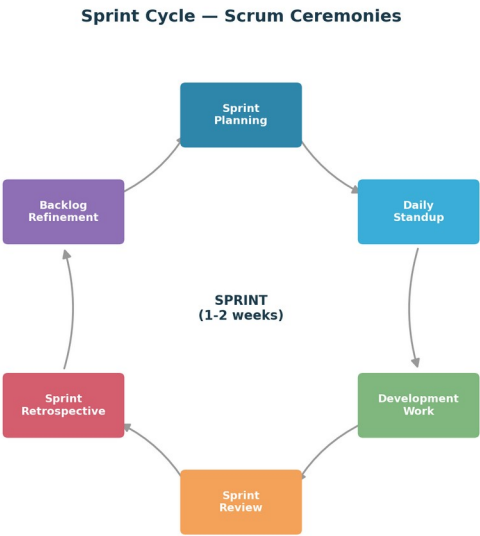


Figure 2: Recurring Scrum ceremonies across a single sprint cycle

### 3. Story Points and Sprint 2 Code Sample

Story points measure relative effort and complexity rather than absolute time, using a modified Fibonacci scale so that estimation differences are meaningful (the gap between a 5 and an 8 forces a real conversation, unlike a gap between 5 and 6). The chart below shows the scale used for every story in this backlog.

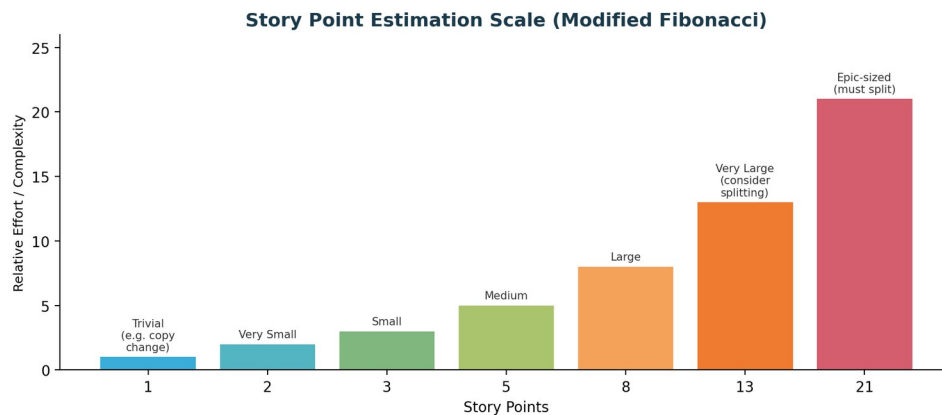


Figure 3: Story point estimation scale (modified Fibonacci) used across all sprints

#### 3.1 Code: Sprint 2 Backlog & Velocity Calculation

The snippet below (Python) models the Sprint 2 backlog pulled from the product backlog table above, and computes planned vs. completed story points and team velocity — the same calculation used at every Sprint Review to decide how much work to commit to in the following sprint. The full script is also provided as a standalone downloadable file.

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class UserStory:
    story_id: str
    title: str
    story_points: int
    status: str = "Not Started" # Not Started | In Progress | Done
    def is_done(self) -> bool:
        return self.status == "Done"

@dataclass
class Sprint:
    name: str
    stories: List[UserStory] = field(default_factory=list)
    def planned_points(self) -> int:
        return sum(s.story_points for s in self.stories)
    def completed_points(self) -> int:
        return sum(s.story_points for s in self.stories if s.is_done())
    def velocity(self) -> float:
        planned = self.planned_points()
        return round((self.completed_points() / planned) * 100, 1) if planned else 0.0

sprint_2 = Sprint("Sprint 2", [
    UserStory("US-04", "Recharge smart card / wallet remotely", 5, "Done"),
    UserStory("US-05", "Station staff gate override with audit log", 3, "Done"),
    UserStory("US-06", "Admin fare-rule configuration screen", 8, "Done"),
    UserStory("US-08", "Refund request before journey start", 5, "In Progress"),
    UserStory("US-09", "Push notification on low wallet balance", 2, "Not Started"),
])

print(f"Planned: {sprint_2.planned_points()} pts | "
      f"Completed: {sprint_2.completed_points()} pts | "
      f"Velocity: {sprint_2.velocity()}%")
# Output: Planned: 23 pts | Completed: 16 pts | Velocity: 69.6%
```

#### 3.2 Summary

Together, the product backlog, the sprint planning snapshot, the story-point scale, and the Sprint 2 velocity calculation give the team a complete, repeatable template: every story is sized consistently, every sprint's commitment is grounded in prior velocity, and progress is measurable in the same units from the first sprint through project closure.